Introduction to Kafka

A Helicopter View...

SAURABH DASHORA

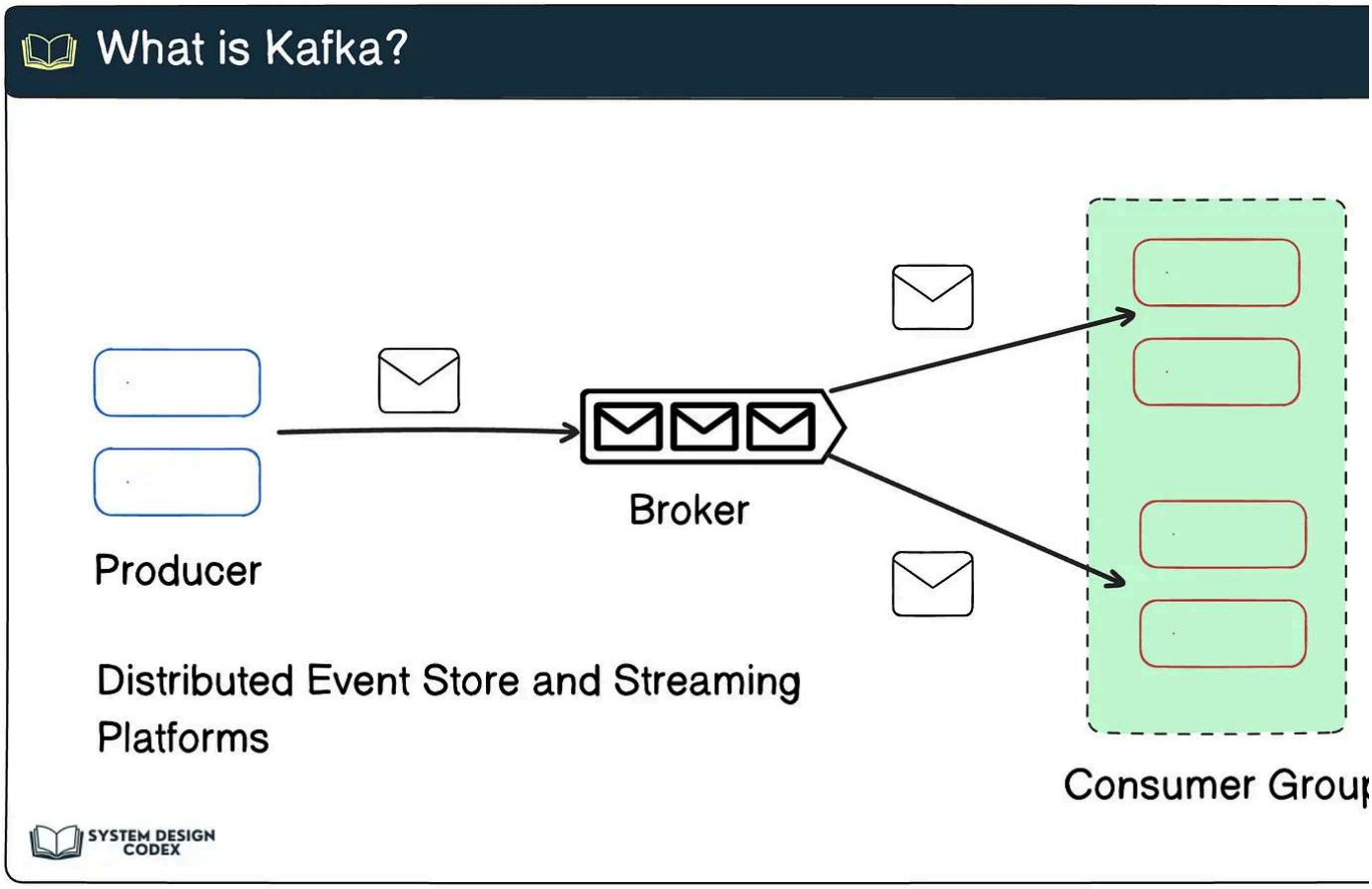
SEP 24, 2024

87 12 7 Sh

Kafka is a distributed event store and a streaming platform.

Over the years, Kafka has become super popular among developers and large organizations.

It began as an internal LinkedIn project but has now become one of the most significant components of event-driven systems. Today, Kafka is used in some of th largest data pipelines in the world. Organizations such as Netflix and Uber rely extensively on Kafka for their data workflows.



You can play around with the diagram on Eraser.io

But what is the reason for this tremendous growth in Kafka's adoption?

The main reasons are reliability and scalability. Kafka can reliably manage the flow data in your application at a huge scale.

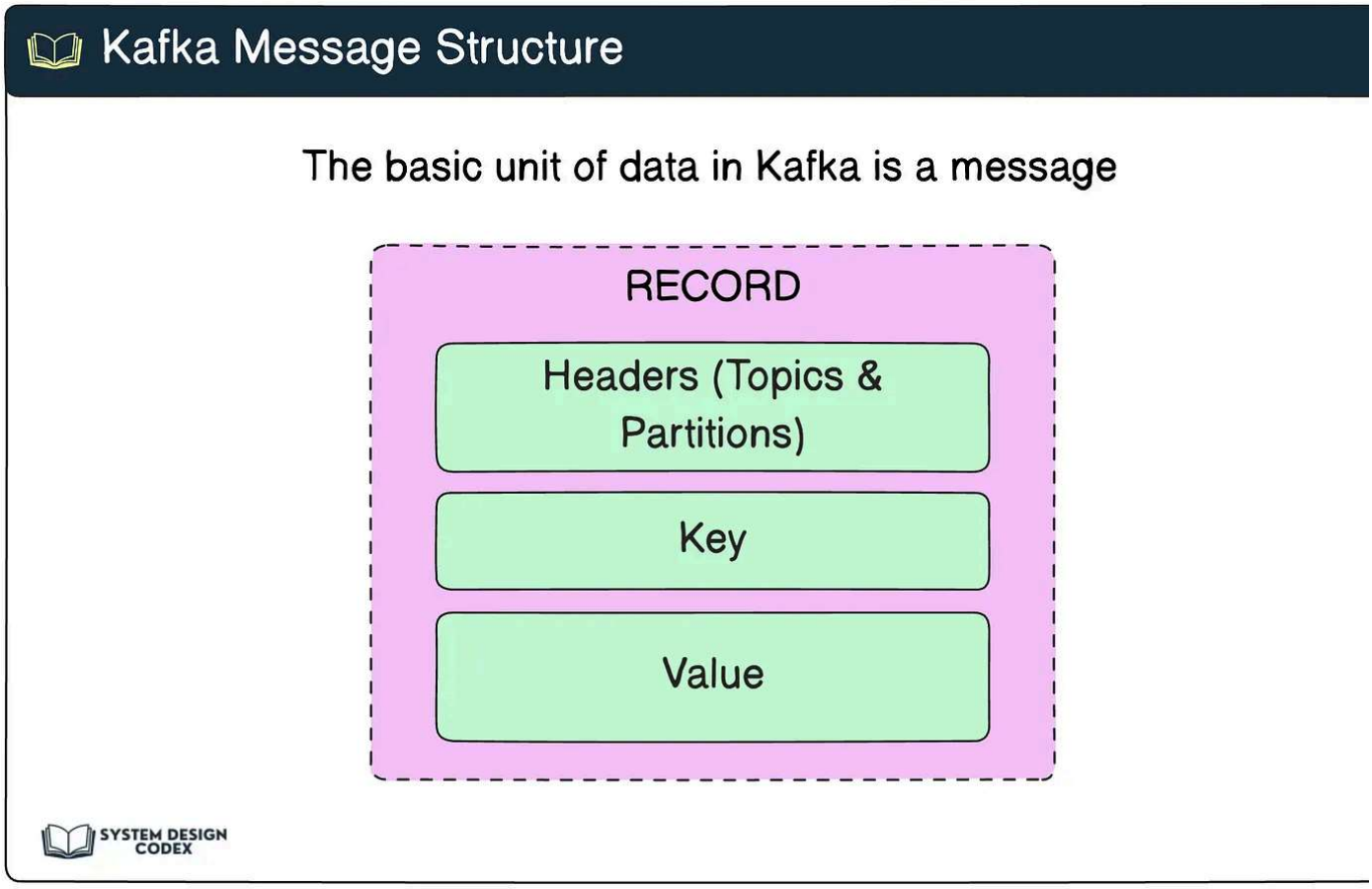
In today's post, we will take a helicopter view of Kafka. In this post, we'll look at the basic building blocks of Kafka.

Kafka Messages and Batches

The basic unit of data in Kafka is called a message.

You can think of a message as a row or record in a database table. However, in the context of Kafka, a message is simply an array of bytes and the data in the message

no specific meaning.



You can play around with the diagram in Eraser.io

Of course, for developers, it is important to have some sort of message structure. W can impose message schemas in Kafka using JSON or XML formats. Kafka favors tl use of another tool known as Apache Avro for managing the schemas.

Messages are written to Kafka in batches. A batch is simply a collection of message produced on the same topic and partition. Hold the two new terms (topic and partition) in your mind as we will get to them in a bit.

But why does Kafka use batches? Why not individual messages?

Because individual round trip of messages is excessive overhead. Collecting messag into a batch reduces this overhead. Batches are also compressed, resulting in efficie

data transfer and storage.

Of course, using batches results in a trade-off between latency and throughput.

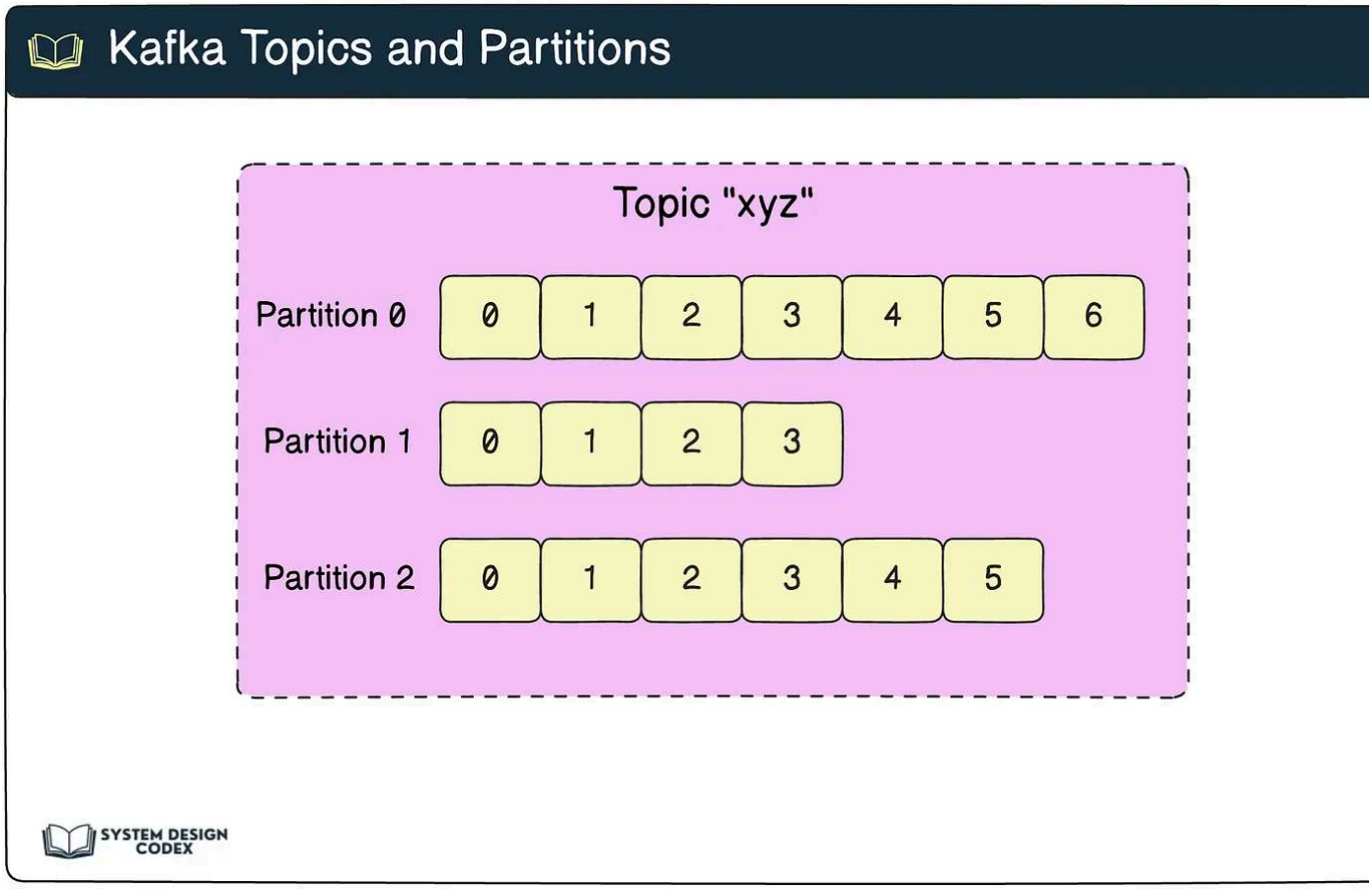
- Larger batch size means more messages per unit of time or increased throughp
- However, it also means longer latency for an individual message. The ideal bat size depends on your specific use case.

Kafka Topics and Partitions

We briefly mentioned these terms in the previous section. They are probably the m important terms for Kafka developers. No Kafka introduction can happen without knowing about topics and partitions. Let us look at them one by one.

> Kafka Topics can be compared to a database table or a folder in a filesystem. Eve message in Kafka is categorized into a particular topic.

Topics are also made up of multiple partitions. See the below diagram where we ha topic named numbers containing three partitions.



You can play around with the diagram on Eraser.io

When we send a message to a Kafka topic, it is appended to a partition. Since the messages are always appended, they can be read in order from beginning to end.

> This guarantee of ordering is only applicable within a particular partition. Since topic can contain multiple partitions, there is no guarantee of message ordering across the entire topic.

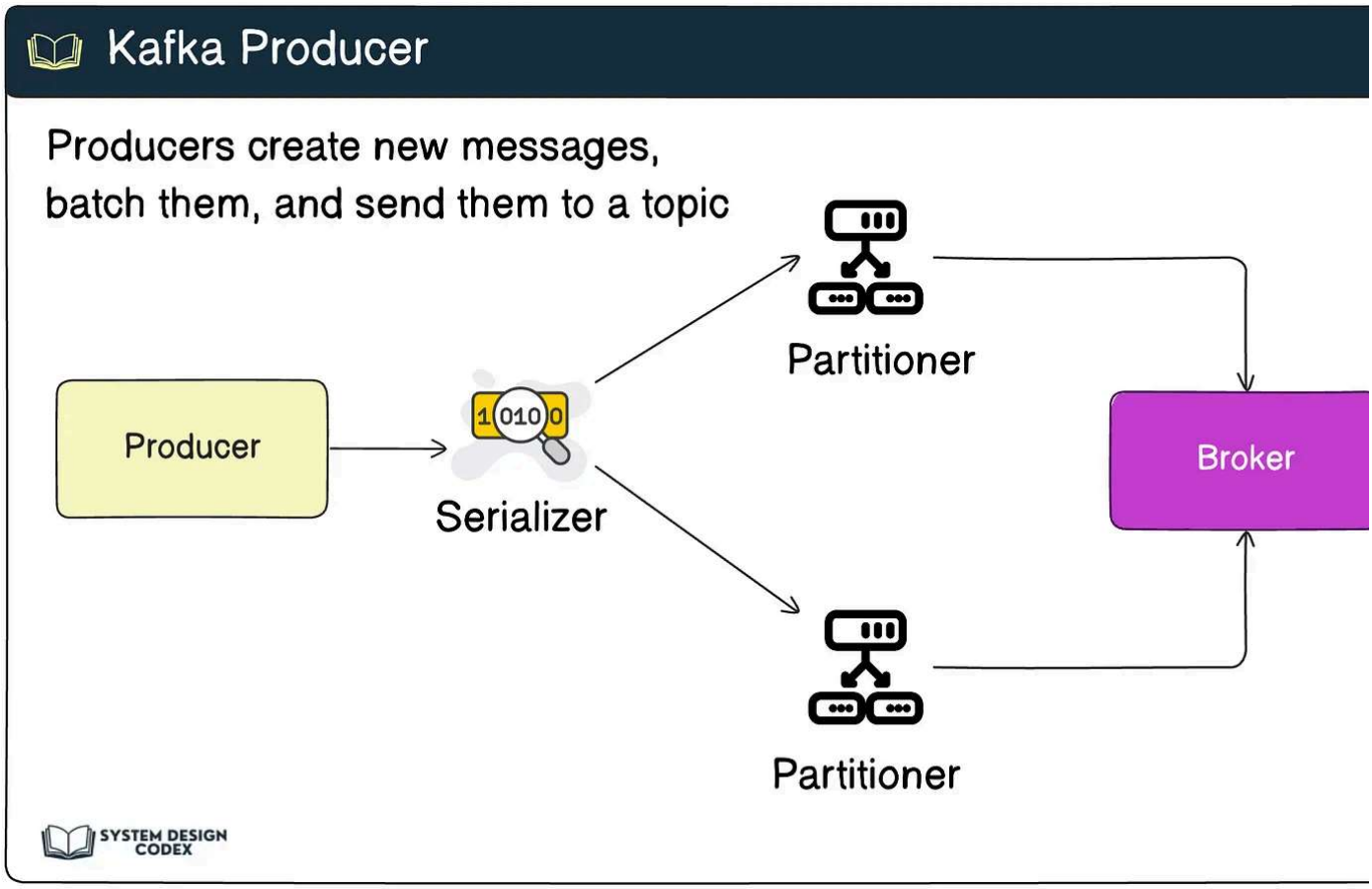
Kafka partitions help provide redundancy and scalability. Each partition can be hos on a different server (or a broker). This makes topics horizontally scalable and can provide performance far beyond the ability of a single server.

Additionally, we can also replicate partitions in Kafka. This means a different serve can store a copy of the same partition. This way, even if a server fails, the partition's data is always available.

Kafka Producers and Consumers

The main users of the Kafka system are also known as Kafka clients. There are two basic types of clients – producers and consumers.

As the name suggests, producers create new messages and send them to a Kafka broker. We also call them publishers or writers.



You can play around with the diagram on Eraser.io

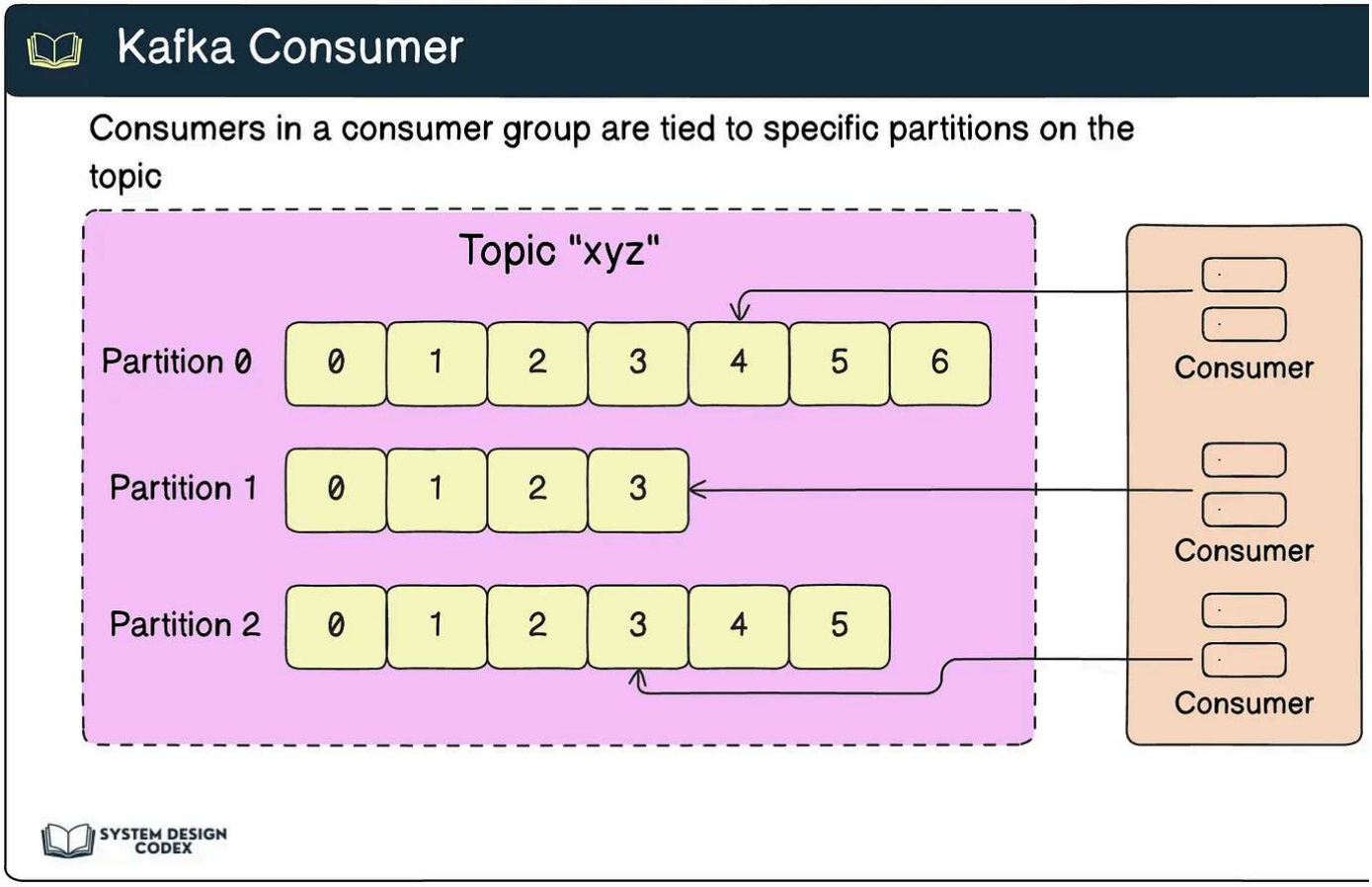
While sending a message, producers specify the topic. By default, producers will balance messages over all partitions of a topic evenly.

However, in some cases, the requirement is to send a particular type of message to a particular partition. Kafka also provides ways to determine the partition by implementing a custom partitioning strategy.

Once the messages are sent to Kafka, it is up to the consumers to read the messages One or more consumers work together as part of a consumer group to consume a topic.

The consumer group ensures that each partition is only consumed by one member. The mapping of a consumer to a partition is often called ownership of the partition the consumer.

See the below diagram of a consumer group consuming messages from partitions. Here, each consumer is responsible for one partition in the topic.



You can play around with the diagram on Eraser.io

Consumers can horizontally scale to consume topics with a large number of messag If a single consumer fails, the remaining members of the group will reassign the partitions being consumed to take over for the missing member.

Kafka Brokers and Clusters

A single Kafka server is called a broker. The job of a Kafka broker can be divided in three parts on the producer side:

- Receive messages from producers
- Assign offset to these messages
- Write messages to storage on the disk

Similarly, it performs the following tasks on the consumer side:

- Handle the fetch requests for partitions
- Respond with the messages that have been published.

A single Kafka broker can handle thousands of partitions and millions of messages second. Moreover, Kafka brokers usually work as part of a cluster.

A Kafka cluster consists of several brokers. One of these brokers plays the role of th cluster controller. The cluster controller is elected automatically from amongst the members of the cluster. This controller is responsible for various administrative operations such as:

- Assigning partitions to brokers
- Monitoring for broker failures

Kafka Cluster Message Replication

The main advantage of a Kafka cluster is the ability to replicate messages. Replicati provides redundancy of messages stored in a partition.

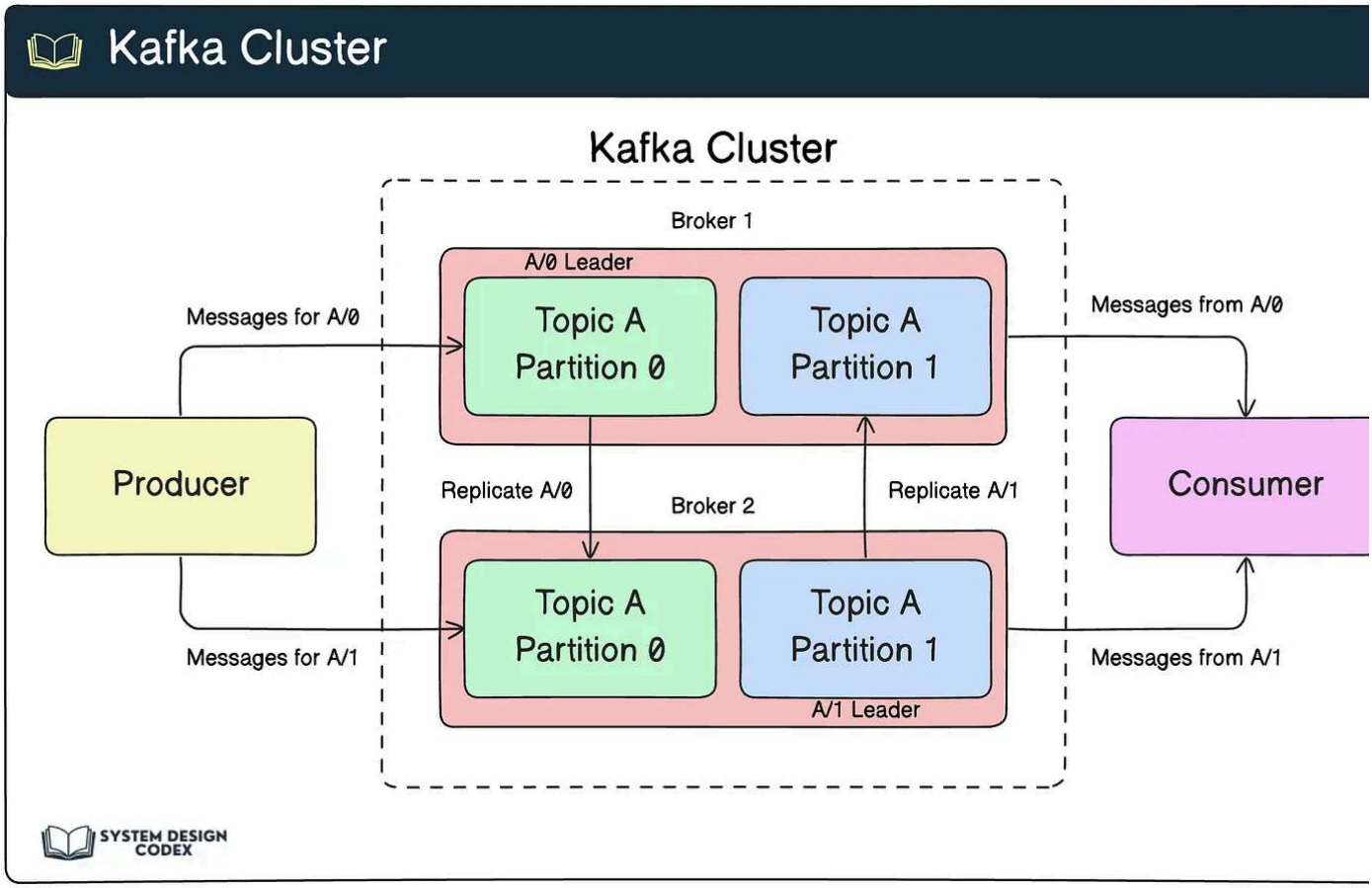
A partition in Kafka is always owned by a single broker in the cluster. This broker i called the leader of the partition. However, when a partition is replicated, it is also

assigned to additional brokers. These additional brokers are followers of the partiti

So, how do producers and consumers interact with a particular partition when ther are several copies available across brokers?

- Producers always connect to the leader broker to publish messages to a particu partition.
- If the leader broker goes down for some reason, one of the follower brokers tak leadership of the partition.
- On the other hand, consumers don't have any such restrictions. They can fetch messages from either the leader or one of the followers.

The below diagram shows the relationship between brokers, topics, and partitions i the context of a Kafka cluster.



You can play around with the diagram on Eraser.io

As you can see, Partition 0 of Topic A is replicated across Broker 1 and Broker 2. However, Broker 1 is the leader of the partition. Similarly, Broker 2 is the leader of Partition 1 of Topic A. The Producer always writes to the leader broker.

The Advantages of Using Kafka

Having understood the high-level components of Kafka, let us look at what makes i special.

- Kafka can handle multiple producers with ease. We can use Kafka to aggregate data from many frontend systems and make it consistent in terms of format. Fo example, a site that serves content to users via several microservices can have a single topic for page views.
- Kafka is designed to support multiple consumers reading a single stream of messages without interfering with each other's client. This is unlike other queu systems where once a message is consumed, it is not available to any other process. On the other hand, multiple Kafka consumers can also join together as group and share a stream. In this case, the entire group processes a message on once.
- Kafka provides disk-based retention. Messages are written to disk and stored w configurable retention rules. This means that consumers can afford to fall behi due to slow processing or spike in traffic. There is no danger of data loss. Anot implication of disk-based retention is the easy maintenance of consumers. If yo need to take down a consumer, there is no concern about messages backing up the producer or getting lost. The messages will be retained in Kafka. Consumer can restart and pick up processing right where they left off with no data loss.
- **Kafka is highly scalable**. This makes it possible for Kafka to handle any amoun data. Users can start with a single broker for initial development and then mov a development cluster of three brokers. In a production setup, we can also have

larger cluster consisting of tens or even hundreds of brokers. Moreover, any expansion in capacity can be done even when the cluster is online without impacting the availability of the system.

Disadvantages of Kafka

Just like everything else in software engineering, Kafka is by no means perfect. Som of the common gripes developers tend to have with Kafka are as follows:

- Kafka provides an overwhelming amount of configuration options. This makes things challenging for new comers as well as seasoned developers in terms of figuring out the optimal settings for the Kafka installation.
- The built-in tooling is sub-par. There is a lack of consistency in the naming of command-line arguments.
- Lack of mature client libraries in languages other than Java or C. The libraries other languages are not up to the mark in terms of quality. However, this is changing fast.
- Lack of true multi-tenancy in terms of completely isolated logical clusters with physical clusters.

So - have you used Kafka in your projects?

Shoutout

Here are some interesting articles I've read recently:

- Did they tell you how to split the database by Raul Junco
- Solving Problems by Sorting by Franco Fernando

- The best way to test Web APIs by Daniel Moka
- SQL vs NoSQL - 7 Key Differences You Must Know by Ashish Pratap Singh
- From Tech Docs to Blog Posts: How Writing Can Transform Your Career by Fran Soto

---

That's it for today! 🌼

Enjoyed this issue of the newsletter?

Share with your friends and colleagues.

See you later with another edition — Saurabh

---

87 Likes · 7 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Anubhav Jain 30 Sept 2024

Liked by Saurabh Dashora

Your explanation is always great . In future posts please consider software architecture patterns a of the topic. The work you are doing simply awesome man!!

LIKE (1) REPLY

1 reply by Saurabh Dashora

Praveen kumar 26 Sept 2024

Liked by Saurabh Dashora

Great content bro

LIKE (1) REPLY

1 reply by Saurabh Dashora

10 more comments...

© 2026 Saurabh Dashora · Privacy · Terms · Collection notice
Substack is the home for great culture